Stingray™ PS225

Quick Start Guide

User Guide

Table of Contents

1 Introduction 5
1.1 Block Diagram 5
1.2 Board Features 6
1.3 Package Contents 7
1.4 PS225 Card Variants 8
2 Checklist 9
3 Initial Board Setup and Connectivity 10
3.1 PS225 Connector Locations 10
3.2 Installing the PS225 10
3.3 Setting Up the Serial Console (Optional) 11
3.4 Accessing the PS225 from an x86 Host 12
3.4.1 Installing L2 Driver on the x86 Host 12
3.4.2 Assigning an IP Address to the First New Host Interface 13
3.4.3 Establishing an SSH Connection to the PS225 16
4 Upgrading the PS225 Software 17
5 Setting Up a Linux Distribution rootfs on the PS225 19
5.1 Booting Different Images 19
5.1.1 Boot the Recovery Image 19
5.1.2 Boot the Factory Default Image 21
5.2 Running Ubuntu on a PS225 23
5.2.1 Setting Up Ubuntu on a PS225 23
5.2.2 Booting the Ubuntu/CentOS/other Distro rootfs Image 29
5.2.3 Verifying the Process 30
5.3 Running CentOS on a PS225 31
5.3.1 Setting Up CentOS on the PS225 31
Appendix A: Repartitioning the eMMC for Larger Data Partitions 36
A.1 Setting Up Files for Repartitioning 36
A.2 Repartitioning eMMC 37
A.2.1 Method 1: Repartition Using a Linux Script 37
A.2.2 Examples of Partition Combinations 39
Appendix B: Dual Redundancy Images Support (DRIS) 40
B.1 DRIS Description and Operation 40
B.2 DRIS Variables 40
B.3 Enabling/Disabling DRIS 41
B.4 Enable/Disable Boot Recovery Service 41
Appendix C: SmartNIC Nitro Configurations 43
C.1 8 + 2 Bare-Metal Configuration 43

C.2 8 + 4 Pairing Configuration 44
C.3 8 + 5 Representor Configuration 45
Appendix D: Frequently Asked Questions 48
Revision History 50

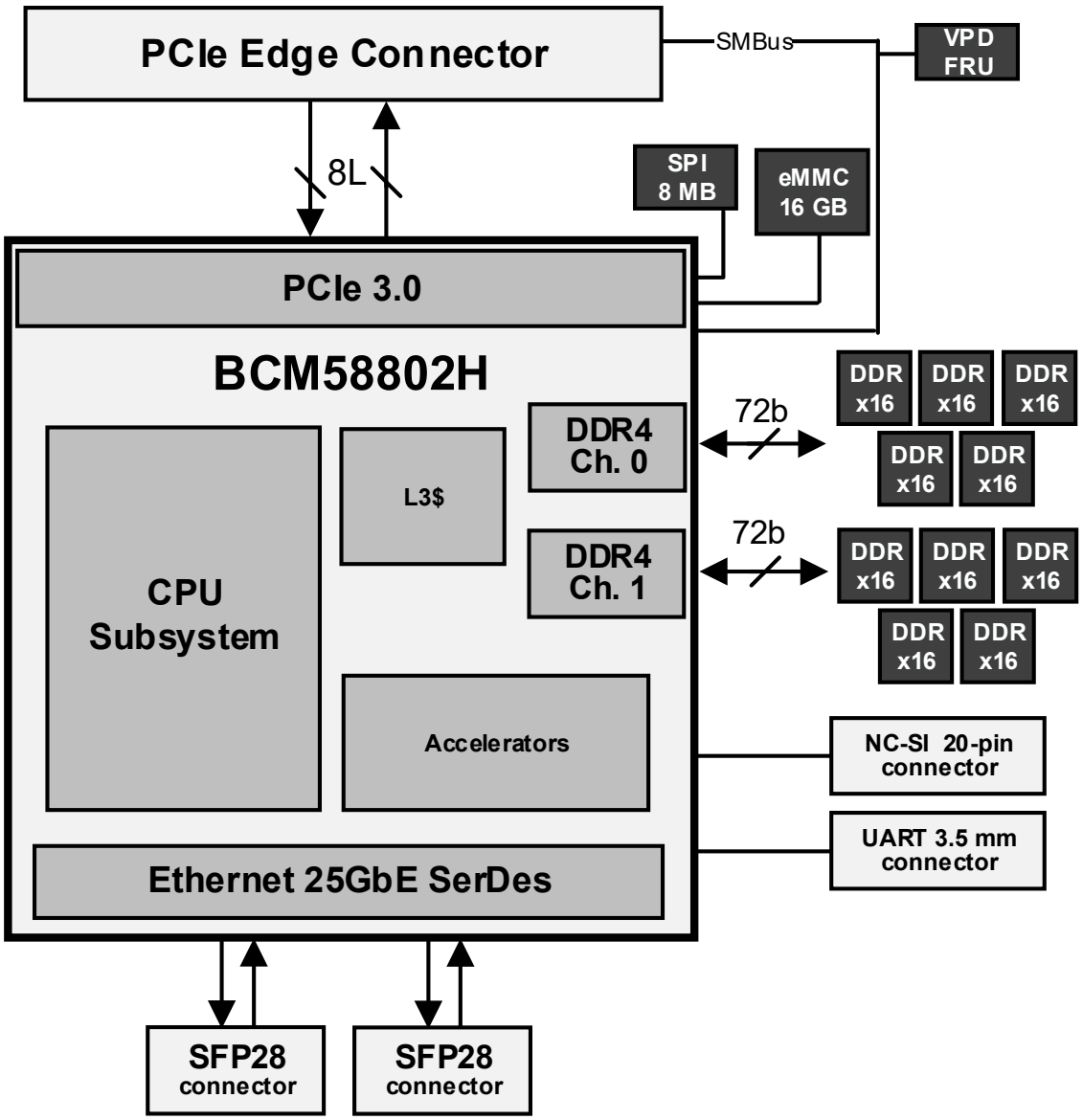
1 Introduction

Broadcom® PS225 SmartNIC adapters, based on Broadcom's latest data center SoC technology, offer groundbreaking network and compute power in a compact package. The PS225 adapters are PCI Express low-profile, half-length adapters, and provide two 25-Gigabit Ethernet (25GbE) ports along with eight 64-bit ARMv8 processors running at an unmatched 3.0 GHz clock rate. The adapters are available in a range of on-board DDR4 memory sizes. The PS225-H04, PS225-H08, and PS225-H16 adapters include 4 GB, 8 GB, and 16 GB of DDR4 memory, respectively.

1.1 Block Diagram

Figure 1 shows the main functional blocks of the PS225 SmartNIC adapter cards.

Figure 1: PS225 Block Diagram



1.2 Board Features

- PCIe Interface:
  - x8 PCI Express 3.0 compliant
  - Single-Root I/O Virtualization (SR-IOV) with up to 128 VFs
  - Function Level Reset (FLR) support
- Processing Subsystem:
  - 8-core ARMv8 A72 64-bit processor subsystem at 3.0 GHz
  - 16 MB total cache (8 MB L2 + 8 MB L3)
  - Two channels (64-bit + ECC) of DDR4 memory at 2,400 MT/s
- Hardware Accelerators:
  - TruFlow™ configurable flow accelerator engine
  - Line-rate Crypto engine with single-pass hashing and encryption
  - RAID 5/6 engine
- Network Interface:
  - Dual-port SFP28 pluggable media interface, supporting 25GbE or 10GbE optical transceiver or direct-attach copper (DAC) cables
  - RDMA over Converged Ethernet (RoCE) v1 and v2
  - Advanced congestion avoidance
  - Virtual network termination – VXLAN, NVGRE, Geneve, GRE
  - Multiqueue, NetQueue, and VMQ
  - Tunnel-aware stateless offloads:
    - IPv4 and IPv6
    - TCP, UDP, and IP checksum
    - Large send offload (LSO)
    - Large receive offload (LRO)
    - TCP segmentation offload (TSO)
    - Receive-side scaling (RSS)
    - Transmit-side scaling (TSS)
    - VLAN insertion/removal
  - DCB support: PFC, ETS, QCN, DCBx
  - Jumbo frames (up to 9 KB)
  - Network boot (PXE, UEFI)
- Security and Manageability:
  - Secure Boot
  - Secure Key Storage
  - ARM PKA Engine
  - NC-SI over MCTP (SMBus, PCIe VDM)
  - NC-SI over RMB (via separate connector)
- Form Factor:
  - PCI Express CEM specification for half-height, half-length adapters
  - I/O brackets available in both low profile and full height

1.3 Package Contents

The PS225 package contains the following components:

- PS225 adapter with low-profile bracket installed
- Full-height bracket

The PS225 card with a low-profile bracket installed is shown in Figure 2.

Figure 2: PS225 with Low-Profile Bracket

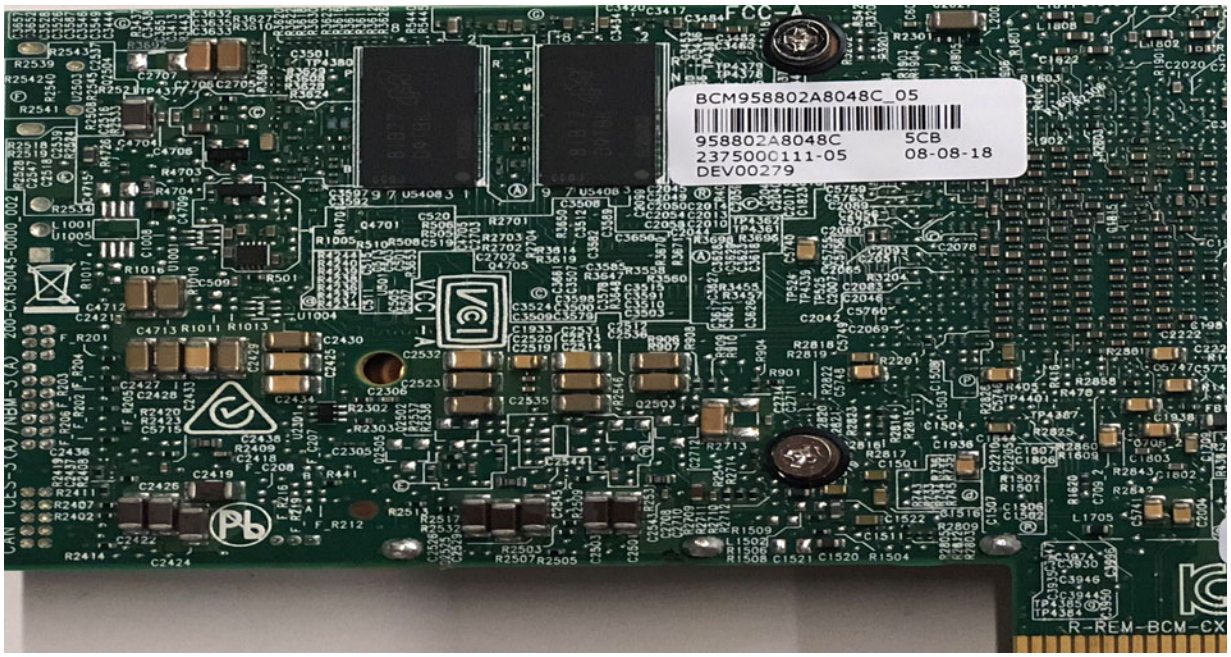
1.4 PS225 Card Variants

The PS225 card comes in the following three variants based on the amount of DDR memory:

- PS225-H04 (4 GB DDR4, part number BCM958802A8024C/BCM958802A8044C)
- PS225-H08 (8 GB DDR4, part number BCM958802A8028C/BCM958802A8048C)
- PS225-H16 (16 GB DDR4, part number BCM958802A8021C/BCM958802A8046C)

The back of the card contains a sticker that indicates the board type. Figure 3 shows an example of the 4G card label on a preproduction adapter.

Figure 3: Example Label



2 Checklist

The following list of major steps must be completed to complete the setup of the PS225:

1. Install the PS225 card in a host machine (see Initial Board Setup and Connectivity).
2. Obtain the documentation listed in Table 1.

Table 1: List of Reference Documentation

Document Name	Document Number	Description
SmartNIC Configuration Guide	5880X-PS225-AN1Xx	Application note for configuring OVS+DPDK on the PS225.
BCM5880X High Performance Ethernet System on Chip	5880X-DS1Xx	BCM5880X data sheet
BCM5880X Hardware Design Guide	5880X-DG1Xx	Hardware design guide
BCM5880X SmartNIC Solution	5880X-UG3Xx	BCM5880X SmartNIC solution
PS225 SmartNIC Adapters Dual-Port 25 Gb/s PCI Express	5880X-PS225-DS1Xx	PS225 advanced data sheet

3. Install the latest binary/image package for the PS225. Upgrade if the PS225 card received has an old image.

NOTE: To check the image version information, refer to docSAFE (the Broadcom documentation portal) to download the binary package required for the specific card type.

```
Package name format : NXS_SN-<release type>_<release version>_<card type>_bin.tar.gz
```

Example: NXS_SN-BM-Beta_1.1.8.0_H08GB_bin.tar

4. Become familiar with the following PS225 basic information:
   - PS225 software version:

     On PS225 Stingray (MAIA) console:

     ```
     root@bcm958802a8048c:~# uname -a
     Linux bcm958802a8048c 4.14.79+g5bb1623 #1 SMP Tue Nov 27 01:48:50 UTC 2018 aarch64
     aarch64 aarch64 GNU/Linux
     ```
   - PS225 board/card type (4 GB, 8 GB, 16 GB):

     On PS225 Stingray (MAIA) console:

     ```
     root@bcm958802a8048c:~# hostname
     bcm958802a8048c
     ```
   - Check the Nitro profile configuration.

     NOTE: By default, the card is upgraded with 2 x 25G port mode and 8 + 8 PFs profile configuration with eight PFs on the Stingray (MAIA) side and eight PFs on the host (x86) side.

Supported Nitro profile configurations in the SmartNIC released image package are described in Appendix C, SmartNIC Nitro Configurations.

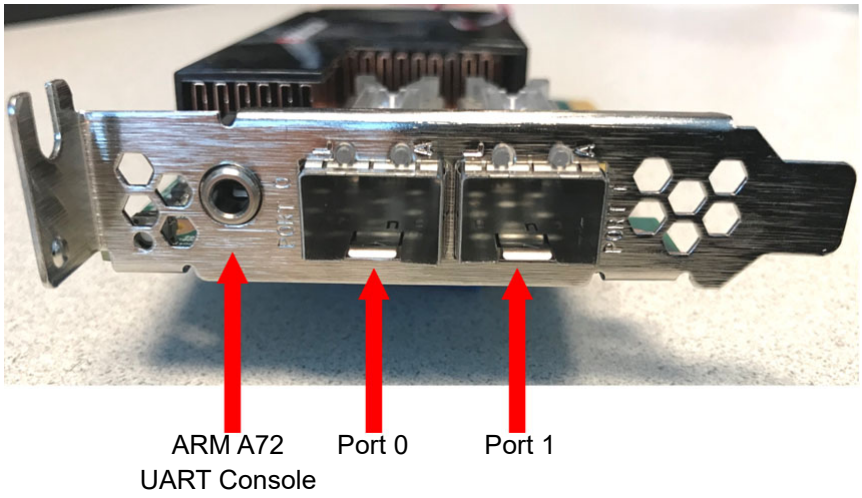
3 Initial Board Setup and Connectivity

This section describes the initial board setup and connectivity.

3.1 PS225 Connector Locations

There are two SFP28 ports and a UART console port for the ARM CPU subsystem located on the I/O panel of the board (see Figure 4).

Figure 4: PS225 Connector Locations



NOTE: The UART console is accessible via the round 3.5 mm jack on the I/O panel. The interface uses TTL-level signaling and can be accessed using a widely available USB-to-TTL cable of type TTL-232R-3V3-AJ (based on FTDI chip).

3.2 Installing the PS225

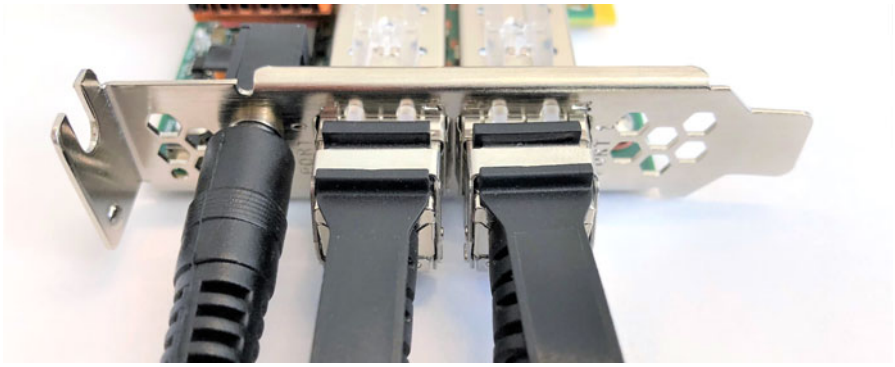
To install the PS225:

1. Power down the host system and remove the AC power.
2. Insert the PS225 card into a PCIe 3.0 slot (at least x8 size) in the host system.
3. Connect one end of the SFP cable to port 0 or port 1 on the PS225 (see Figure 5).
4. Connect the other end of the SFP cable to an Ethernet switch that supports 10G or 25G Ethernet (see Figure 5).

NOTE: Both Ethernet ports must be connected to link partners operating at the same speed: either both 25GbE or both 10GbE. The ports do not support mixing 25GbE and 10GbE links.

5. Optionally:
   a. Connect the other port of the PS225 card to an Ethernet switch (see Figure 5).
   b. Insert the 3.5 mm plug of a USB serial cable to the 3.5 mm socket on the front panel (see Figure 5).

Figure 5: Connecting the PS225 Cables



3.3 Setting Up the Serial Console (Optional)

To set up the optional serial console:

1. Using a USB serial 3.3V to 3.5 mm cable (specified in the following link), insert the 3.5 mm plug into the card and the USB plug into a PC that has the terminal emulator program installed.

   https://www.digikey.com/product-detail/en/ftdi-future-technology-devices-international-ltd/TTL-232R-3V3-AJ/768-1098-ND/2285997

2. Install the appropriate UART-USB driver:
   - For a Windows OS, the driver for the UART-USB transceiver is downloaded from the FTDI driver download site (2.12.28, http://www.ftdichip.com/Drivers/CDM/CDM21228_Setup.zip).
   - For a Linux OS, the device is automatically enumerated as one of the USB tty (`/dev/ttyUSBx`).
   - If another operating system is used, see the FTDI website for available drivers.
3. Access the ARM A72 console using a terminal emulator and configure the UART as follows:
   - Baud rate: 115200
   - Data: 8-bit
   - No parity
   - Stop bit
   - No flow control

   A typical Linux setup for Minicom is shown in the following screen capture:

```
+-----------------------------------------------------------------------+
| A -    Serial Device      : /dev/ttyUSB0                              |
| B - Lockfile Location     : /var/lock                                 |
| C -   Callin Program      :                                           |
| D -  Callout Program      :                                           |
| E -    Bps/Par/Bits       : 115200 8N1                                |
| F - Hardware Flow Control : No                                        |
| G - Software Flow Control : No                                        |
|                                                                       |
|    Change which setting?                                              |
+-----------------------------------------------------------------------+
```

3.4 Accessing the PS225 from an x86 Host

NOTE: The instructions in this document assume that the GA 1.0.2 (or later) SmartNIC software release is preloaded on the PS225. Contact Broadcom support regarding the software version that was delivered on the card. If a card is delivered with older software, contact Broadcom support for upgrade procedures. The upgrade procedure described in this document does not apply to releases earlier than GA 1.0.2.

When the PS225 boots up for the very first time, it assigns a fixed IP address of 192.168.1.10 to the first PF on the ARM side.

The x86 host can access this IP address after the following actions:

1. Install appropriate Layer 2 (L2) bnxt driver.
2. Assign IP address on the new x86 network interface.
3. Verify that the interface is up and check the link status (`ip link show` or `ifconfig`).
4. Establish an SSH connection from the x86 host to the SSH server running on the PS225 ARM Linux.

NOTE: The initial IP address of the first interface on the PS225 card (192.168.1.10) can be changed by modifying `/etc/network/interfaces` using either the serial console or after the first successful ssh to the card. The change in `/etc/network/interfaces` takes effect on the next reboot of the card. If the IP address is changed during the active ssh session, ensure that the ssh reconnect is to the new IP address.

3.4.1 Installing L2 Driver on the x86 Host

The L2 driver source code can be provided as part of the SmartNIC release or as a separate release package. Before running the following script, install the packages in advance that have a dependency with L2 Driver:

```
sudo apt-get install cmake libpci-dev
```

1. If the driver is provided as part of the release package, run the provided install script.

   Example:

   ```
   ./poky/images/bcm958802a8021/x86-host-install.sh
   ```

   This script creates a subdirectory `brcm-nxt-l2-drv` where the driver is compiled. The current version of the install script intentionally omits installing the driver as part of the kernel. An `insmod` must be used instead of `modprobe`, but prior to using `insmod`, the dependency modules must be installed.

   Example:

   ```
   sudo modprobe ptp
   sudo modprobe vxlan
   sudo modprobe devlink

   # from the top-level directory above poky:

   sudo insmod brcm-nxt-l2-drv/git/main/Cumulus/drivers/linux/v3/bnxt_en.ko
   ```

2. If the driver is provided separately, it must be extracted and compiled for the kernel running on the x86 host and installed. Example commands are provided below:

NOTE: It is assumed that the delivered L2 source code is in the tarball `brcm-nxt-l2-drv.tar.gz`.

```
tar zxf brcm-nxt-l2-drv.tar.gz
cd git/main/Cumulus/drivers/linux/v3/
make
sudo make install
```

The driver name is `bnxt_en.ko` and the last command installs it in the system in the `/lib/modules/<kernel-version>/updates/` directory.

3. Install the driver using the following command:

```
sudo modprobe bnxt_en
```

3.4.2 Assigning an IP Address to the First New Host Interface

After a `modprobe` or `insmod` of the `bnxt_en` driver, the new network interfaces show up on the x86 host. The number of interfaces/PFs that show up on the x86 host and separately on the ARM side is Nitro firmware-dependent. An example for x86 is shown below and the interfaces are **highlighted**.

NOTE: The exact names may differ from system to system and depend on the OS being used and the physical slot where the card is installed.

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen
1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s31f6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode DEFAULT group
default qlen 1000
    link/ether b0:6e:bf:30:44:65 brd ff:ff:ff:ff:ff:ff
19: enp1s0f0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default
qlen 1000
    link/ether 00:10:18:de:06:08 brd ff:ff:ff:ff:ff:ff
20: enp1s0f1d1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:09 brd ff:ff:ff:ff:ff:ff
21: enp1s0f2: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0a brd ff:ff:ff:ff:ff:ff
22: enp1s0f3d1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0b brd ff:ff:ff:ff:ff:ff
23: enp1s0f4: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0c brd ff:ff:ff:ff:ff:ff
24: enp1s0f5d1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0d brd ff:ff:ff:ff:ff:ff
25: enp1s0f6: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0e brd ff:ff:ff:ff:ff:ff
26: enp1s0f7d1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc mq state DOWN mode DEFAULT group
default qlen 1000
    link/ether 00:10:18:de:06:0f brd ff:ff:ff:ff:ff:ff
```

1. Assign an IP address on the 192.168.1.x subnet to the interface for function 0 (in the previous example, it is enp1s0f0).

   Example:

   ```
   sudo ifconfig enp1s0f0 192.168.1.20 up

   # OR

   sudo ip addr add 192.168.1.20/24 dev enp1s0f0
   ```

2. Check the new IP address with either an `ifconfig` or `ip addr show` command. The PS225 (the ARM host on the PS225) can now be pinged.

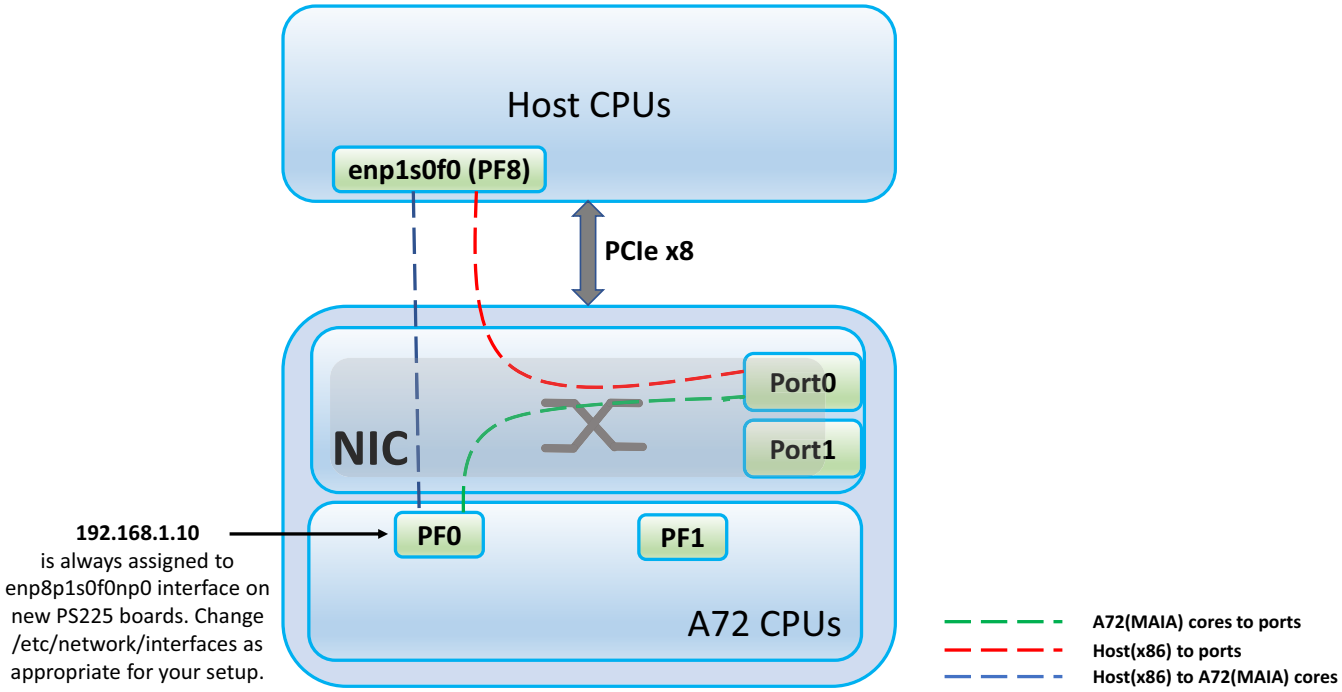
   Example:

   ```
   $ ping 192.168.1.10
   PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
   64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.131 ms
   64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.046 ms
   ...
   ```

Figure 6 shows the connectivity of the first interface on the x86 host and the first interface on the ARM side. 192.168.1.20 is assigned to enp1s0f0 (on the x86 host) and 192.168.1.10 to enP8p1s0f0np0 on the ARM host on the PS225. These two interfaces are connected through Nitro (red arrow).

NOTE: As shown in Figure 6, each interface can send packets out on the 25GbE port (blue arrow for the x86 enp1s0f0 host interface and green arrow for the ARM enP8p1s0f0np0 interface).

Figure 6: Interface Connectivity



When sending packets to the external ports, the even-numbered interfaces on the x86 host (for example, enp1s0f0, enp1s0f2, enp1s0f4, and enp1s0f6) are connected to external port 0. This also applies to the even-numbered interfaces on the ARM side (enP8p1s0f0np0, enP8p1s0f2np0, enP8p1s0f4np0, and enP8p1s0f6np0). The odd-numbered interfaces on both the x86 host side and the ARM side are connected to external port 1.

Use one of the following options to permanently assign a static IP address to the interface:

- For Debian-based Linux OS (for example, Ubuntu) set a static IP address in `/etc/network/interface` with the following commands:

```
auto enp1s0f0

iface enp1s0f0 inet static
        address 192.168.1.20
        netmask 255.255.255.0
```

- For CentOS, create a new file using the following commands:

```
/etc/sysconfig/network-scripts/ifcfg-ens4f0
```

NOTE: The previous example is provided for an interface named `ens4f0.` The interface name may be different on different systems.

Add the following content:

```
TYPE="Ethernet"
PROXY_METHOD="none"
BROWSER_ONLY="no"
BOOTPROTO="static"
DEFROUTE="yes"
NAME="ens4f0"
UUID="660e94eb-a5b4-4975-8139-388675307f7d"
DEVICE="ens4f0"
ONBOOT="no"
IPADDR=192.168.1.20
NETMASK=255.255.255.0
GATEWAY=192.168.1.20
```

The content can be completely reused, but the **highlighted** items must be replaced. The variables NAME and DEVICE contain the interface name that was determined above. The variable UUID contains the ID of the interface, which can be obtained by running `uuidgen ens4f0`.

3.4.3 Establishing an SSH Connection to the PS225

To establish an SSH connection to the PS225:

Assuming that the PS225 is available (through ping), the x86 host can access the ARM host on the PS225 by running SSH using the following command:

```
ssh root@192.168.1.10
```

No password is required. The following command line prompt displays:

```
$ ssh root@192.168.1.10
Last login: Fri Feb 16 22:19:21 2018
root@bcm958802a8048c:~#
```

During the first run, it is recommended to execute `uname -a` to verify that the latest kernel available for the PS225 is being used (verify the latest kernel with the Broadcom support team).

4 Upgrading the PS225 Software

The upgrade procedure for PS225 relies on an external TFTP or web server. Normally this server is the machine that hosts the PS225, but it can also be an external server on the network. The binary image must be placed in a subdirectory on the TFTP/web server.

To upgrade the PS225 software:

1. From the host SSH to the PS225 using the following command:

```
ssh root@192.168.1.10
```

2. Run the update script with the following commands:

```
root@bcm958802a8048c:~# update-me.sh -s <serverip> -d <dir> -i <image> -t <transport>
-s <serverip>  IPv4 address of http or tftp server
-d <dir>       Directory under the top-level TFTP or HTTP server directory
-i <image>     Image can take one of the following values (or maybe newer ones)
                 all       Copies all files, dtb, fip, kernel, nitro and rootfs
                 dtb       Copies dtb only
                 fip       Copies fip only
                 kernel    Copies fip kernel only
                 nitro     Copies nitro firmware only
                 nonitro   Copies all but nitro
                 noroot    Copies fip, dtb and kernel, nitro, no rootfs
                 rootfs    Copies rootfs only
 -t <transport> Transport (tftp or http) (default tftp)
 -u <url>       URL link to image tarball. This will use 'wget' and will bypass tftp/http.
 -n <nitro_image>  Nitro Image to use (default nitro)
 -c <nitro_config> Nitro Config to use (default none = do not update config)
 -r <yes/no/off>   Reboot (yes) or Poweroff (off) or nothing after upgrade (default no)
 -f <file> file to download and run (default updateme.sh)
```

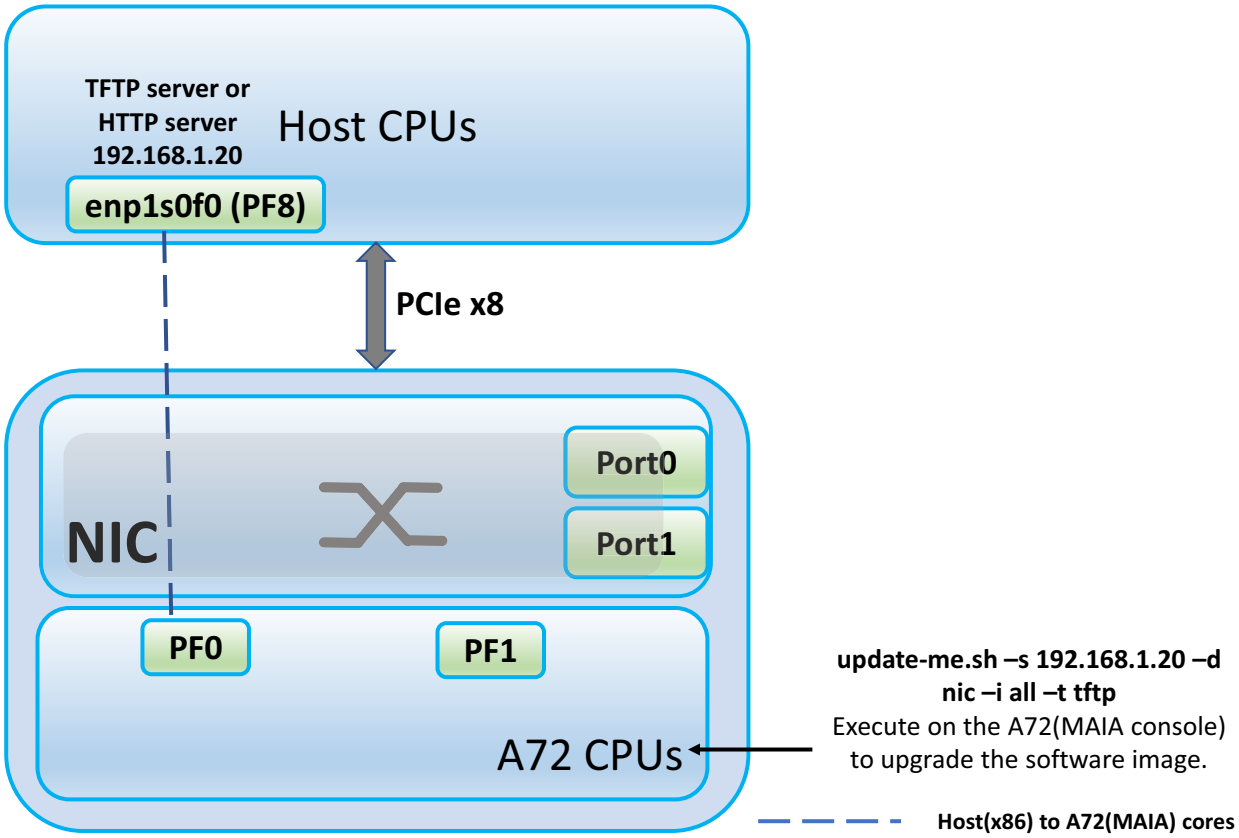
Example:

```
root@bcm958802a8048c:~# update-me.sh -s 192.168.1.20 -d nic -i all -t tftp
```

3. Reboot the PS225 once the image update has completed. The reboot command must be applied on the ARM side.

NOTE: If the reboot or poweroff command is applied on the x86 host after the PS225 update, ensure that the L2 driver `bnxt_en` is removed (`sudo modprobe -r bnxt_en` command on the x86 host side) so that the resources on the host are free.

Figure 7: Upgrading the PS225 Software



5 Setting Up a Linux Distribution rootfs on the PS225

This section describes how to set up a Linux distribution rootfs on the PS225.

5.1 Booting Different Images

Understand the eMMC partitions and images flashed on each partition before making any changes to the eMMC partitions for flashing Ubuntu/CentOS/other distro rootfs.

eMMC is partitioned by default into five partitions as shown in Table 2.

Table 2: eMMC Partitions

Partition name	Size	Purpose	Comment/Remarks
mmcblk0boot0 / mmcblk0boot1	N/A	UEFI firmware	fip.bin is flashed into this partition.
mmcblk0p1	524288	Kernel and dtb images. Images are represented with slot number(id)	Kernel image and dtb corresponds to image set #1 and set #2.
mmcblk0p2	786432	Yocto Recovery rootfs. Slot ID is R	During the update-me.sh image upgrade procedure, slot which is not active gets flashed.
mmcblk0p3	4194304	Yocto rootfs. Slot id is 1	During the update-me.sh image upgrade procedure, MMC partition 4 gets updated with new rootfs if current Linux boots up with this (mmc partition 3) partition.
mmcblk0p4	4194304	Yocto rootfs. Slot id is 2	During the update-me.sh image upgrade procedure, mmc partition 3 gets updated with new rootfs if current Linux boots up with this (mmc partition 4) partition.
mmcblk0p5	2097152	Free	Partition can be used for other distro rootfs. For example, Ubuntu or CentOS.

To boot into a different set of images if there is a change in the default flashing or upgrade procedure:

5.1.1 Boot the Recovery Image

This set of images are useful to recover the card if a card is not fully functional for basic network connectivity. Stop the SmartNIC from booting up in the UEFI during the firmware boot up. On a serial console, press any key when *Press any key within 5 seconds* displays. On UEFI shell, use the following steps:

```
Shell> select
Select slot type not specified. Valid slot types are kernel, dtb, rootfs, rootfs-ordinal.
Currently active slots:
kernel: 2
dtb   : 2
rootfs: 2
Shell> select kernel
 -------- kernel IMAGE SELECTION -------
   1. kernel.1
 * 2. kernel.2
 -------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot
```

```
Please enter menu option:
> R
OS is instructed to boot into recovery.
Shell> select dtb
 -------- dtb IMAGE SELECTION -------
   1. dtb.1
 * 2. dtb.2
 ==== Available dtb sources ====
   3. FS2:\bcm958802a804x.dtb
 ==== 1 dtb source(s) found ====
 ------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot

Please enter menu option:
> R
OS is instructed to boot into recovery.
FS2:\> select rootfs
 -------- rootfs IMAGE SELECTION -------
   1. LinuxRoot.1
 * 2. LinuxRoot.2
 ------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot

Please enter menu option:
> R
OS is instructed to boot into recovery.
FS2:\> startup auto
 set autoboot yes
bootcfg=EMMC
 esp_root=FS2:
Notice: ELOG is enabled.
Nitro SIT version: SIT_rel_nxt_x.x.x
Number of linux boot attempts: 3
Forcing booting into Recovery Mode as requested
Found LinuxRoot recovery partition, partition ordinal 2.
READ-ONLY access forced.
 --- RECOVERY SCENE PREPARED.
earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000

FS2:\> startup boot
bootcfg=EMMC
 esp_root=FS2:
Notice: ELOG is enabled.
Nitro SIT version: SIT_rel_nxt_x.x.x
Number of linux boot attempts: 3
Forcing booting into Recovery Mode as requested
Found LinuxRoot recovery partition, partition ordinal 2.
READ-ONLY access forced.
 --- RECOVERY SCENE PREPARED.
earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000
bootcmd=FS2:\Image  root=/dev/mmcblk0p2 ro rootwait  earlycon=uart8250_log,mmio32,0x68A00000
earlylo
```

```
g=0x8f110000,0x10000 console=ttyS1,115200n8 hugepages=0 cma=1G crashkernel=512M pci=pcie_bus_safe
pc
iehp.pciehp_poll_mode=1 pciehp.pciehp_poll_time=5 pcie_ports=native maxcpus=8     loglevel=
Starting Recovery/Legacy Image
```

5.1.2 Boot the Factory Default Image

This set of images are the default (or upgraded) images that the card must use to boot up without any issues. If any issues with this set of images are observed, see Boot the Recovery Image to recover the card.

Stop the SmartNIC from booting up in the UEFI during the firmware boot up. On a serial console, press any key when *Press any key within 5 seconds* displays. On UEFI shell, use the steps shown in Table 3.

Table 3: Booting the Factory Default Image

Boot into Slot Number 1 Set Of Images	Boot into Slot Number 2 Set Of Images
Shell> **select kernel** -------- kernel IMAGE SELECTION ------- 1. kernel.1 * 2. kernel.2 --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 1** Slot 1 is selected for kernel image. Shell> **select dtb** -------- dtb IMAGE SELECTION ------- 1. dtb.1 * 2. dtb.2 ==== Available dtb sources ==== 3. FS2:\bcm958802a804x.dtb ==== 1 dtb source(s) found ==== --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 1** Slot 1 is selected for dtb image. FS2:\> **select rootfs** -------- rootfs IMAGE SELECTION ------- 1. LinuxRoot.1 * 2. LinuxRoot.2 --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 1** Slot 1 is selected for rootfs image. FS2:\> **startup auto** set autoboot yes bootcfg=EMMC esp_root=FS2: Notice: ELOG is enabled. Nitro SIT version: SIT_rel_nxt_214_v214.2.251.0 Number of linux boot attempts: 5 set root root=/dev/mmcblk0p3 earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000 FS2:\> **startup boot** bootcfg=EMMC esp_root=FS2: Notice: ELOG is enabled. Nitro SIT version: SIT_rel_nxt_214_v214.2.251.0 Number of linux boot attempts: 5 set root root=/dev/mmcblk0p3 earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000 bootcmd=FS2:\Image.1 root=/dev/mmcblk0p3 rw rootwait earlycon=uart8250_log,mmio32,0x68A00000 early log=0x8f110000,0x10000 console=ttyS1,115200n8 hugepages=0 cma=1G crashkernel=512M pci=pcie_bus_safe pciehp.pciehp_poll_mode=1 pciehp.pciehp_poll_time=5 pcie_ports=native maxcpus=8 loglevel= Kernel slot 1, dtb slot 1, rootfs slot 1	Shell> **select kernel** -------- kernel IMAGE SELECTION ------- 1. kernel.1 * 2. kernel.2 --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 2** Slot 2 is already selected for kernel image. Shell> **select dtb** -------- dtb IMAGE SELECTION ------- 1. dtb.1 * 2. dtb.2 ==== Available dtb sources ==== 3. FS2:\bcm958802a804x.dtb ==== 1 dtb source(s) found ==== --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 2** Slot 2 is already selected for dtb image. FS2:\> **select rootfs** -------- rootfs IMAGE SELECTION ------- 1. LinuxRoot.1 * 2. LinuxRoot.2 --------------------------------------- R. Recovery Q. Quit * denotes currently selected slot **Please enter menu option:** **> 2** Slot 2 is selected for rootfs image. FS2:\> **startup auto** set autoboot yes bootcfg=EMMC esp_root=FS2: Notice: ELOG is enabled. Nitro SIT version: SIT_rel_nxt_214_v214.2.251.0 Number of linux boot attempts: 4 set root root=/dev/mmcblk0p4 earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000 FS2:\> **startup boot** bootcfg=EMMC esp_root=FS2: Notice: ELOG is enabled. Nitro SIT version: SIT_rel_nxt_214_v214.2.251.0 Number of linux boot attempts: 4 set root root=/dev/mmcblk0p4 earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000 bootcmd=FS2:\Image.2 root=/dev/mmcblk0p4 rw rootwait earlycon=uart8250_log,mmio32,0x68A00000 early log=0x8f110000,0x10000 console=ttyS1,115200n8 hugepages=0 cma=1G crashkernel=512M pci=pcie_bus_safe pciehp.pciehp_poll_mode=1 pciehp.pciehp_poll_time=5 pcie_ports=native maxcpus=8 loglevel= Kernel slot 2, dtb slot 2, rootfs slot 2

5.2 Running Ubuntu on a PS225

This section describes a basic Ubuntu 16.04 setup on a PS225 board. It is assumed that an x86 host with Ubuntu 16.04 is being used. With small changes, it should apply to other variants as well.

NOTE: Follow Upgrading the PS225 Software and successfully boot to Linux from the most recent SmartNIC release on the PS225.

5.2.1 Setting Up Ubuntu on a PS225

5.2.1.1 Installing Ubuntu 16.04/CentOS 7.4 with a Script

There are existing scripts that can be used to install Ubuntu 16.04 and CentOS 7.4 on the eMMC. By default, these scripts install the OS on `/dev/mmcblk0p5` on the eMMC.

The two scripts are located in rootfs 1 (`/dev/mmcblk0p3`) and rootfs 2 (`/dev/mmcblk0p4`) under the `/usr/share/edk2` directory. The script to install Ubuntu can be run as `/usr/share/edk2/yocto2ubuntu.sh` and the script to install CentOS can be run as `/usr/share/edk2/yocto2centos.sh`.

The two scripts automate the majority of the manual installation steps described in Installing Ubuntu 16.04/CentOS 7.4 Manually. The scripts also instruct the UEFI to use the updated partition for boot, as described in Booting the Ubuntu/CentOS/other Distro rootfs Image, therefore, the next boot after running these scripts is to `/dev/mmcblk0p5` on which Ubuntu/CentOS has been installed.

5.2.1.2 Installing Ubuntu 16.04/CentOS 7.4 Manually

To set up Ubuntu on a PS225:

1. SSH to the PS225 card or use the serial console to access the PS225 Linux shell.
2. Create a new ext4 partition on the eMMC card. Ensure that the existing partitions used for UEFI (and yocto) are not deleted.
3. Run `fdisk -l` to verify the current partitions on `/dev/mmcblk0`.
   If mmcblk0p5 is already created and it is 2 GB, skip Step 3.

NOTE: mmcblk0p5 may have already been created during installation of the SmartNIC release.

NOTE: In future releases, the partition layout may change. Check the release documentation before proceeding with Step 4 and Step 5.

4. Use the `fdisk` command to create `/dev/mmcblk0p5`. Skip this step if it has already been completed. Refer to the following example command:

```
# boot normally, so from Yocto built kernel and RootFS

root@bcm958802a8048c:~# fdisk /dev/mmcblk0
   type 'n' to create a new partition
   used defaults (will created mmcblk0p5)
   typed 'w' to write changes
```

5. Format the partition using the following command:

```
root@bcm958802a8048c:~# mkfs.ext4 /dev/mmcblk0p5
```

NOTE: This step takes several minutes.

6. Mount the newly created partition to a directory of choice (for example, /mnt) using the following commands:

```
root@bcm958802a8048c:~# mkdir /mnt/ubuntu
root@bcm958802a8048c:~# mount /dev/mmcblk0p5 /mnt/ubuntu
```

7. Download the latest 16.04 ARM64 image from Ubuntu to the x86 host and untar it on the PS225. See the following commands:

NOTE: Ensure that there is connectivity with the host (follow the *PS225 Quick Start Guide* in Table 1 for additional details).

a. On the host, download the Ubuntu image with the following commands:

```
byang@byang-PowerEdge-2950:~$ wget http://cloud-images.ubuntu.com/releases/16.04/release/ubuntu-16.04-server-cloudimg-arm64-root.tar.xz
```

b. Ensure connectivity between host and PS225 with the following commands (see the *PS225 Quick Start Guide* in Table 1):

```
byang@byang-PowerEdge-2950:~$ sudo insmod ~/temp/bnxt_en-x.x.x/bnxt_en.ko
byang@byang-PowerEdge-2950:~$ sudo ifconfig enp8s0f0 192.168.1.20 up ; ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:
Reply from 192.168.1.10: bytes=32 time=1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

c. On the PS225, scp the Ubuntu image with the following command:

```
root@bcm958802a8048c:~# scp byang@192.168.1.20:~/ubuntu-16.04-server-cloudimg-arm64-root.tar.xz.
```

d. Set the correct date with the following commands so that tar accepts the future time.

```
root@bcm958802a8048c:~# date -s "14 MAR 2018 20:00:00"

root@bcm958802a8048c:~# cd /mnt/ubuntu
root@bcm958802a8048c:/mnt/ubuntu# tar --numeric-owner -xf /mnt/ubuntu/ubuntu-16.04-server-cloudimg-arm64-root.tar.xz
```

8. The new rootfs is extracted in the specified partition (in the example above `/dev/mmcblk0p5`). `chroot` to the device to set up several items before the kernel boots that rootfs for the first time. This step is important, otherwise the Ubuntu OS does not boot properly. See the following commands:

a. Copy the bnxt* modules from the yocto filesystem to the ubuntu rootfs with the following command:

```
root@bcm958802a8048c:/mnt/ubuntu# cp -r /lib/modules/$(uname -r) lib/modules
```

b. Switch to Ubuntu's filesystem through chroot with the following command:

```
root@bcm958802a8048c:/mnt/ubuntu# chroot /mnt/ubuntu
```

c. Remove cloud init from this prebuilt image (since it interferes with normal boot) with the following command:

```
\u@\h:\w$ dpkg --purge cloud-init
```

d. Add getty for ttyS0 with the following commands:

```
\u@\h:\w$ cp /etc/init/ttyAMA0.conf /etc/init/ttyS0.conf
\u@\h:\w$ sed 's/ttyAMA0/ttyS0/g' -i /etc/init/ttyS0.conf
```

e. Set up the user lab with the following commands:

```
\u@\h:\w$ adduser lab
```

f. Enter the user name and password information.

g. Add the user lab to admin and sudo groups with the following commands:

```
\u@\h:\w$ addgroup lab admin
\u@\h:\w$ addgroup lab sudo
\u@\h:\w$ exit
```

9. Unmount the device, reboot, and stop at the UEFI prompt:

```
root@bcm958802a8048c:/mnt/ubuntu# cd /mnt
root@bcm958802a8048c:/mnt# umount /mnt/ubuntu
root@bcm958802a8048c:/mnt# reboot
```

10. The system is now ready to boot the new Ubuntu rootfs, however, the new rootfs to be used by kernel must be specified. The UEFI must use Ubuntu rootfs for boot. The UEFI then sets up Linux args automatically. See the following commands:

```
Press any key within 5 seconds
AUTOBOOT SKIPPED
Shell> gpt list blk0
Shell> select rootfs-ordinal 5
Shell> select
Shell> startup
```

The login prompt and username/password that was set up above is now available. This may take several minutes to get to the login prompt. See the following commands:

NOTE: It is possible to switch back to yocto Linux by performing the following using the UEFI.

```
Shell> select rootfs-ordinal 3
Shell> startup
```

11. Acquire network connectivity with host using the following commands:

NOTE: If bnxt_en.ko module in yocto Linux has previously insmod'ed, power cycle and try again.

a. On the PS225 console, (optionally) get rid of the warning "sudo: unable to resolve host ubuntu: Connection refused" by editing /etc/hosts, and add the following line (in blue):

```
lab@ubuntu:/$  sudo nano /etc/hosts
127.0.0.1 localhost
127.0.1.1 ubuntu
```

b. Continue to load the driver and setup the network interface as follows:

```
lab@ubuntu:~$ sudo depmod -a # apply only at the very first boot
lab@ubuntu:~$ sudo modprobe bnxt_en
lab@ubuntu:~$ ifconfig -a
```

Note the name of the interface slot that is plugged into the PS225.

```
lab@ubuntu:/$ sudo ifconfig enP8p1s0f0np0 up 192.168.1.10
```

c. On the host, insmod the bnxt_en.ko module. See the PS225 Quick Start Guide to find out where to get/build this module for the x86 host. The following command operates on the example setup.

```
byang@byang-PowerEdge-2950:~$ sudo insmod ~/temp/bnxt_en-1.8.29/bnxt_en.ko
byang@byang-PowerEdge-2950:~$ sudo ifconfig enp8s0f0 192.168.1.20 up ; ping 192.168.1.10
```

d. Access between the host and PS225 should be now available.

12. Allow the PS225 to reach the Internet, so the user can use apt install to retrieve software from the Internet. We configure the host to forward using the following commands:

NOTE: If your PS225's SFP28 port is connected to the Internet (for example, via a switch and DHCP server), then there is no need for host to forward Internet traffic. Skip the following steps.

NOTE: "eno2" is the regular Ethernet interface that the host uses to access the Internet while enp8s0f0 is the first PF towards the PS225 card. On different machines, these could be different. Adjust the commands based on the specific setup.

a. On the example host, the result from ifconfig is as follows:

```
byang@byang-PowerEdge-2950:~/temp/bnxt_en-1.8.29$ ifconfig
eno2      Link encap:Ethernet  HWaddr 00:22:19:91:7c:e4
          inet addr:10.67.93.199  Bcast:10.67.93.255  Mask:255.255.254.0
          inet6 addr: fe80::116e:caa:13f3:1dbf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:23392 errors:0 dropped:0 overruns:0 frame:0
          TX packets:16471 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2509414 (2.5 MB)  TX bytes:2926607 (2.9 MB)

enp8s0f0  Link encap:Ethernet  HWaddr 00:10:18:ad:05:08
          inet addr:192.168.1.20  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::210:18ff:fead:508/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1661 errors:0 dropped:0 overruns:0 frame:0
          TX packets:344 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
```

```
          RX bytes:481235 (481.2 KB)  TX bytes:32437 (32.4 KB)

enp8s0f2  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0a
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1183 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1321 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:381443 (381.4 KB)  TX bytes:271592 (271.5 KB)

enp8s0f4  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0c
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1186 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1307 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:382936 (382.9 KB)  TX bytes:267866 (267.8 KB)

enp8s0f6  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0e
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1174 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1315 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:378828 (378.8 KB)  TX bytes:270156 (270.1 KB)

enp8s0f1d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:09
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:189 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1312 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:65394 (65.3 KB)  TX bytes:270596 (270.5 KB)
enp8s0f3d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0b
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

enp8s0f5d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0d
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

enp8s0f7d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0f
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2002 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2002 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:170071 (170.0 KB)  TX bytes:170071 (170.0 KB)
```

b. On the host (assuming that it is Ubuntu 16), ensure that it is changed based on the distribution.

c. Edit `/etc/sysctl.conf` and uncomment the following line:

```
net.ipv4.ip_forward=1
```

```
byang@byang-PowerEdge-2950:~$ sudo iptables -t nat -A POSTROUTING -o eno2 -j MASQUERADE
byang@byang-PowerEdge-2950:~$ sudo iptables -A FORWARD -i eno2 -o enp8s0f0 -m state --state
RELATED,ESTABLISHED -j ACCEPT
byang@byang-PowerEdge-2950:~$ sudo iptables -A FORWARD -i enp8s0f0 -o eno2 -j ACCEPT
```

d. On PS225, configure default gateway and DNS server with the following commands:

```
lab@ubuntu:/$ sudo route add default gw 192.168.1.20
lab@ubuntu:/$ sudo nano /etc/resolvconf/resolv.conf.d/head
```

e. Add the following line to /etc/resolvconf/resolv.conf.d/head:

```
nameserver 8.8.8.8
lab@ubuntu:/$ sudo resolvconf -u
```

f. Test it by pinging Google:

```
lab@ubuntu:/$ ping google.com
```

g. Install something useful with the apt command:

```
lab@ubuntu:/$ sudo apt update
Install gcc, for example.
lab@ubuntu:/$ sudo apt install gcc
lab@ubuntu:~$ sudo su
root@ubuntu:~$ sed 's/PasswordAuthentication/# PasswordAuthentication/g' -i /etc/ssh/sshd_config
root@ubuntu:/home/lab# dpkg-reconfigure openssh-server
```

h. On the host, SSH from the host to the PS225 using the following command:

```
byang@byang-PowerEdge-2950:~$ ssh lab@192.168.1.10
```

5.2.2 Booting the Ubuntu/CentOS/other Distro rootfs Image

This set of images are excellent images to receive the benefit of the distro's repos for the additional software installation and other development work on a target directly. If any issues with this set of images are observed, see Boot the Recovery Image to recover the card.

Stop the SmartNIC from booting up in the UEFI during the firmware boot up. On a serial console, press any key when *Press any key within 5 seconds* displays. On UEFI shell, use the following steps:

```
Shell> select kernel
 -------- kernel IMAGE SELECTION -------
 * 1. kernel.1
   2. kernel.2
 -------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot

Please enter menu option:
> 1
Slot 1 is already selected for kernel image.
Shell> select dtb
 -------- dtb IMAGE SELECTION -------
 * 1. dtb.1
   2. dtb.2
 ==== Available dtb sources ====
   3. FS2:\bcm958802a804x.dtb
 ==== 1 dtb source(s) found ====
 -------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot

Please enter menu option:
> 1
Slot 1 is already selected for dtb image.
FS2:\> select rootfs-ordinal 5
Partition Data (ordinal 5) is set as current rootfs.
FS2:\> select
Select slot type not specified. Valid slot types are kernel, dtb, rootfs, rootfs-ordinal.
Currently active slots:
kernel: 1
dtb   : 1
rootfs: 1 (not used for boot) | 5 (custom ordinal - used for boot), partition name: Data
FS2:\> startup auto
 set autoboot yes
bootcfg=EMMC
 esp_root=FS2:
Notice: ELOG is enabled.
Nitro SIT version: SIT_rel_nxt_x.x.x
Number of linux boot attempts: 21
set root root=/dev/mmcblk0p5
earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000
FS2:\> startup boot
bootcfg=EMMC
```

```
 esp_root=FS2:
Notice: ELOG is enabled.
Nitro SIT version: SIT_rel_nxt_x.x.x
Number of linux boot attempts: 21
set root root=/dev/mmcblk0p5
earlycon=uart8250_log,mmio32,0x68A00000 earlylog=0x8f110000,0x10000
bootcmd=FS2:\Image.1  root=/dev/mmcblk0p5 rw rootwait  earlycon=uart8250_log,mmio32,0x68A00000
early
log=0x8f110000,0x10000 console=ttyS1,115200n8 hugepages=0 cma=1G crashkernel=512M
pci=pcie_bus_safe
pciehp.pciehp_poll_mode=1 pciehp.pciehp_poll_time=5 pcie_ports=native maxcpus=8     loglevel=
Kernel slot 1, dtb slot 1, rootfs Data (custom, ordinal 5)
```

5.2.3 Verifying the Process

Any of the three previous methods must be performed carefully with the understanding of the partitions and the selection of the image set.

Use the following process to confirm if the compatible image set for the system has booted correctly. If the image does not match, reboot the SmartNIC and select a different rootfs using the `select rootfs` command (see Table 3).

- `uname -a` shows exact kernel version

  ```
  root@bcm958802a8044c:~# uname -a
  Linux bcm958802a8044c 4.14.95 #1 SMP Tue Feb 26 16:58:59 UTC 2019 aarch64 aarch64 aarch64 GNU/Linux
  ```

- `ls /lib/modules/` shows the directory name matching the kernel version. This dertermines if the kernel modules in the currently mounted rootfs partition are correct.

  ```
  root@bcm958802a8044c:~# ls /lib/modules/
  4.14.95
  ```

- `mount` shows the MMC partition that is mounted as rootfs.

  ```
  root@bcm958802a8044c:~# mount
  /dev/mmcblk0p3 on / type ext4 (rw,relatime,data=ordered)
  devtmpfs on /dev type devtmpfs (rw,relatime,size=1210424k,nr_inodes=302606,mode=755)
  sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
  proc on /proc type proc (rw,relatime)
  ```

5.3 Running CentOS on a PS225

This section describes basic CentOS 7 setup on a PS225 board. It is assumed that an x86 host with Ubuntu 16.04 is being used. With small changes, it should apply to other variants as well.

NOTE: Follow Upgrading the PS225 Software and successfully boot to Linux from the most recent SmartNIC release on the PS225.

5.3.1 Setting Up CentOS on the PS225

To set up CentOS on the PS225:

1. SSH to the PS225 card or use the serial console to access the PS225 Linux shell.
2. Create a new ext4 partition on the eMMC card.

CAUTION! Ensure that the existing partitions used for UEFI (and Yocto) are not deleted.

If mmcblk0p5 is already created and it is 2 GB, skip Step 3. mmcblk0p5 is already created during the installation of GA 1.0.3.

NOTE: In future releases, the partition layout may change. Check the release documentation before proceeding with Step 4 and Step 5.

3. Run `fdisk -l` to verify the current partitions on `/dev/mmcblk0`.
4. Create `/dev/mmcblk0p5` with the following `fdisk` command. Skip this step if this has already been done.

```
root@bcm958802a8048c:~# fdisk /dev/mmcblk0
   type 'n' to create a new partition
   used defaults (will created mmcblk0p5)
   typed 'w' to write changes
```

5. Format the partition with the following command:

```
root@bcm958802a8048c:~# mkfs.ext4 /dev/mmcblk0p5
```

NOTE: This step can take several minutes to complete.

6. Mount the newly created partition to a directory of choice with the following command (for example. /mnt):

```
root@bcm958802a8048c:~# mkdir /mnt/centos
root@bcm958802a8048c:~# mount /dev/mmcblk0p5 /mnt/centos
```

7. Download the latest 16.04 ARM64 image from Centos to the x86 host and untar it on the PS225. Ensure that there is connectivity with the host (see the PS225 Quick Start Guide for additional details).
   a. Download the image with the following command:

```
byang@byang-PowerEdge-2950:~$wget http://vault.centos.org/altarch/7.4.1708/isos/aarch64/CentOS-7-aarch64-rootfs-7.4.1708.tar.xz
```

NOTE: This link can be changed at any time. See http://mirror.centos.org/.

   b. Ensure connectivity with the host and PS225 with the following commands:

```
byang@byang-PowerEdge-2950:~$ sudo insmod ~/temp/bnxt_en-1.8.29/bnxt_en.ko
byang@byang-PowerEdge-2950:~$ sudo ifconfig enp8s0f0 192.168.1.20 up ; ping 192.168.1.10
```

c. On the PS225, SCP the CentOS image with the following command:

```
cd /mnt/centos
root@bcm958802a8048c:/mnt/centos# scp byang@192.168.1.20:~/CentOS-7-aarch64-rootfs-7.4.1708.tar.xz
root@bcm958802a8048c:/mnt/centos# tar --numeric-owner -xf CentOS-7-aarch64-rootfs-7.4.1708.tar.xz
```

d. Free up space with the following command:

```
root@bcm958802a8048c:/mnt/centos# rm CentOS-7-aarch64-rootfs-7.4.1708.tar.xz
```

8. With the new rootfs extracted in the specified partition (in the example, `/dev/mmcblk0p5`), chroot to the device to set up the device before the kernel boots that rootfs for the first time. This step is important, otherwise the Ubuntu OS will not boot properly.

a. Copy the bnxt* modules from the Yocto filesystem to the ubuntu rootfs with the following command:

```
cp -r /lib/modules/`uname -r` lib/modules
```

b. Switch to CentOS's filesystem through chroot with the following commands:

```
chroot /mnt/centos
adduser lab
passwd lab
gpasswd -a lab wheel
exit
cd
umount /mnt/centos
reboot
```

9. Unmount the device and stop at the UEFI prompt:

NOTE: At this point, the new CentOS rootfs is ready to boot, however, the new rootfs to be used by the kernel must be set. Instruct UEFI to use the CentOS rootfs for boot. The UEFI sets up Linux args automatically.

```
Press any key within 5 seconds
AUTOBOOT SKIPPED
Shell> gpt list blk0
Shell> select rootfs-ordinal 5
Shell> select
Shell> startup
```

At this point, it is possible to reach the login prompt and log in with the username/password that was created in the previous steps. It may take a few minutes to reach the login prompt.

NOTE: It is possible to switch back to Yocto Linux by using the UEFI.

```
select rootfs-ordinal 3
startup
```

10. Provide connectivity to the host using the following steps:

a. Install modules copied in Step 8, Step a.

NOTE: If the insmod'ed bnxt_en.ko module in Yocto Linux has been previously installed, power cycle the system and try again.

b. On the PS225 console, run the following commands:

```
[lab@localhost ~]$ sudo depmod -a # apply only at the very first boot
[lab@localhost ~]$ sudo modprobe bnxt_en.
```

11. On the PS225, edit `/etc/sysconfig/network-scripts/ifcfg-enP8p1s0f0np0` and add the following lines:

```
DEVICE= enP8p1s0f0np0
GATEWAY=192.168.1.20
IPADDR=192.168.1.10
NETMASK=255.255.255.0
DNS1=8.8.8.8
[lab@localhost ~]$ sudo ifup enP8p1s0f0np0
[lab@localhost ~]$ sudo ip addr show
```

a. On the host, insmod to `bnxt_en.ko` module using the following commands (see the PS225 Quick Start Guide for information on building this module on an x86 host):

```
sudo insmod ~/temp/bnxt_en-1.8.29/bnxt_en.ko
sudo ifconfig enp8s0f0 192.168.1.20 up ; ping 192.168.1.10
```

b. The ping to the PS225 board should be successful. SSH and SCP between the host and the PS225 with the following command:

```
# ssh lab@192.168.1.10
```

12. All the PS225s must be able to access the Internet so that the `yum install` command can be used to retrieve the software from the Internet.

NOTE: If the PS225's SFP28 port is connected to the Internet (for example, via a switch and DHCP server), there is no need for the host to forward internet traffic. Skip the following steps.

The results of ifconfig on the host are as follows:

NOTE: Eno2 is the regular Ethernet interface that the host uses to access the Internet while enp8s0f0 is the first PF towards the PS225 card. On different systems these items could be different. Adjust these items based on your setup.

```
byang@byang-PowerEdge-2950:~/temp/bnxt_en-1.8.29$ ifconfig
eno2      Link encap:Ethernet  HWaddr 00:22:19:91:7c:e4
          inet addr:10.67.93.199  Bcast:10.67.93.255  Mask:255.255.254.0
          inet6 addr: fe80::116e:caa:13f3:1dbf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:23392 errors:0 dropped:0 overruns:0 frame:0
          TX packets:16471 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2509414 (2.5 MB)  TX bytes:2926607 (2.9 MB)

enp8s0f0  Link encap:Ethernet  HWaddr 00:10:18:ad:05:08
          inet addr:192.168.1.20  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::210:18ff:fead:508/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1661 errors:0 dropped:0 overruns:0 frame:0
          TX packets:344 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:481235 (481.2 KB)  TX bytes:32437 (32.4 KB)
```

```
enp8s0f2  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0a
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1183 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1321 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:381443 (381.4 KB)  TX bytes:271592 (271.5 KB)

enp8s0f4  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0c
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1186 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1307 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:382936 (382.9 KB)  TX bytes:267866 (267.8 KB)

enp8s0f6  Link encap:Ethernet  HWaddr 00:10:18:ad:05:0e
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1174 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1315 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:378828 (378.8 KB)  TX bytes:270156 (270.1 KB)

enp8s0f1d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:09
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:189 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1312 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:65394 (65.3 KB)  TX bytes:270596 (270.5 KB)
enp8s0f3d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0b
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

enp8s0f5d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0d
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

enp8s0f7d1 Link encap:Ethernet  HWaddr 00:10:18:ad:05:0f
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:437 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:151202 (151.2 KB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2002 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2002 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:170071 (170.0 KB)  TX bytes:170071 (170.0 KB)
```

13. On the host (assume it is Ubuntu 16), ensure that the change is based on your distribution.
14. Edit `/etc/sysctl.conf` and uncomment the following line:

```
net.ipv4.ip_forward=1

byang@byang-PowerEdge-2950:~$ sudo iptables -t nat -A POSTROUTING -o eno2 -j MASQUERADE
byang@byang-PowerEdge-2950:~$ sudo iptables -A FORWARD -i eno2 -o enp8s0f0 -m state --state
RELATED,ESTABLISHED -j ACCEPT
byang@byang-PowerEdge-2950:~$ sudo iptables -A FORWARD -i enp8s0f0 -o eno2 -j ACCEPT
```

15. Test the configuration by accessing www.google.com.

```
lab@ubuntu:/$ ping google.com
```

Appendix A: Repartitioning the eMMC for Larger Data Partitions

This appendix provides information on repartitioning the eMMC for larger data partitions.

A.1 Setting Up Files for Repartitioning

To set up files for repartitioning:

1. Extract the binary images from the release tarball (`NXS_SN-<release type>_<release version>_<card type>_bin.tar.gz`) to a folder in the `/var/lib/tftpboot` directory on the TFTP server.
2. View the part.txt file in the images folder and modify the partition sizes. For example, allocate 7 GB to data partition used for loading Ubuntu rootfs:

```
#part.txt file content

ShellCommand,""
Part,"ESP","512","0","1","1","EFI System"
Part,"LinuxRoot","768","19","0","0","Recovery"
Part,"LinuxRoot.1","1536","12","0","0","Rootfs.1"
Part,"LinuxRoot.2","1536","12","0","0","Rootfs.2"
Part,"Data","7168","7","0","0","Data"
Name,"Broadcom-Default"
Count,"5"
Rootfs,"2"
```

3. After changing the partition sizes in part.txt file, ensure that the md5sum of part.txt is updated in the md5sum.txt file.

```
root@ubuntu:/var/lib/tftpboot/nic# md5sum part.txt
229b92f2586e3fc0c1d9a62b4f9f7f10  part.txt

#Open 'md5sum.txt' in the images folder and replace md5sum of 'part.txt' with the new md5sum
obtained above and save the file

root@ubuntu:/var/lib/tftpboot/nic# vi md5sum.txt
MD5 sums for the output files
229b92f2586e3fc0c1d9a62b4f9f7f10 ./part.txt
.
.
259963d951abc4e69c82610c40b0c5bb  ./dtbs.tar
```

NOTE: The full content of md5sum.txt is not displayed in the previous example.

A.2 Repartitioning eMMC

This section describes two methods for repartitioning eMMC and examples of partition combinations.

A.2.1 Method 1: Repartition Using a Linux Script

To repartition using a Linux script:

1. From the UEFI shell on the PS225, boot to recovery mode and repartition the eMMC partitions using the following commands:

```
shell> select rootfs
-------- rootfs IMAGE SELECTION -------
    * 1. LinuxRoot.1
    2. LinuxRoot.2
-------------------------------------
   R. Recovery
   Q. Quit

* denotes currently selected slot

Please enter menu option:
> R
```

2. Type **R** when prompted.

```
shell> startup boot
```

   The system boots into recovery mode.

   An alternative way of entering recovery mode that does not require the user to stop at the UEFI prompt is to force the recovery mode on the next reboot from Linux:

```
root@bcm958802a8044:~# cd /usr/share/edk2
root@bcm958802a8044:/usr/share/edk2# ./force_recovery.sh && reboot
```

3. In Linux, use the `repartition.sh` script to repartition eMMC with the following command:

```
root@bcm958802a8044:~# cd /usr/share/edk2
```

4. Open the `repartition.sh` script and ensure that the correct inputs have been given for serverip and dir using the following script:

```
repartition.sh -s <serverip> -d <dir> -t <transport>
Helper script, used to flash one or more of the requested images to emmc.
A reboot, after flashing, must be performed for the new images to take effect.
 -s <serverip>    IPv4 address of http or tftp server
 -d <dir>         Directory under the top-level TFTP or HTTP server directory
 -t <transport>   Transport (tftp or http)

root@bcm958802a8044:/usr/share/edk2# ./repartition.sh -s <serverip> -d <image_folder_name> -t
<tftp/http>
```

5. After the script execution is complete, upgrade rootfs so the repartition takes effect using the following command:

   ```
   root@bcm958802a8044:/usr/share/edk2# update-me.sh -s <serverip> -d <image_folder_name> -i rootfs -
   t tftp
   ```

   When the script is running, make a note of the rootfs partition to which the new rootfs is flashed:

   ```
   ************ RECOVERY MODE DETECTED ***************
   Updating ROOTFS...
   rootfs_slot=1
   rootfs_partition=3
   ```

6. After flashing is complete, power cycle the system for new images to take effect. Stop in UEFI and change rootfs slot to the one where rootfs is flashed (red text in the previous step).

   ```
   shell> select rootfs
   (select slot ‘1’ and hit ENTER)
   shell> startup boot
   ```

   The system boots to Linux with the new images.

A.2.2 Examples of Partition Combinations

Table 4 provides examples of partition combinations.

Table 4: Examples of Partition Combinations

Description	P1 (Name and Size)	P2 (Name and Size)	P3 (Name and Size)	P4 (Name and Size)	P5 (Name and Size)	Count	Rootfs
Dual partition (default – two rootfs partitions)	EFI System – 512	Recovery – 768	Rootfs.1 – 4096	Rootfs.2– 4096	Data – 2048	5	2
Dual partition (example – smaller rootfs partitions, larger data partition)	EFI System – 512	Recovery– 768	Rootfs.1 – 2048	Rootfs.2 – 2048	Data – 4096	5	2
Dual partition (example – different size for rootfs partitions)	EFI System – 512	Recovery – 768	Rootfs.1 – 5120	Rootfs.2 – 2048	Data – 2048	5	2
Single partition (example)	EFI System – 512	Recovery – 768	Rootfs.1 – 4096	Data – 2048	N/A	4	1

Appendix B: Dual Redundancy Images Support (DRIS)

This section describes the behavior of the PS225 (Stingray SmartNIC) card booting up into the recovery image partition or the redundant image partition due to DRIS feature support. To detect that the card has booted up with the recovery image, check the output of command `mount` from the MAIA Linux shell if partition `/dev/mmcblk0p2` is mounted at /.

NOTE: By default, `mmcblk0p2` is flashed with the recovery image rootfs.

To disable the recovery image boot, see Enable/Disable Boot Recovery Service.

NOTE: The recovery image boot has limited functionality. The host can SSH to the MAIA default IP address: 192.168.1.10 to access the MAIA Linux shell to take recovery steps.

B.1 DRIS Description and Operation

DRIS represents a set of identical mirrors for several potentially independent images: dtb, kernel, and rootfs. The number of mirrors is arbitrary and is restricted only by the capabilities/partitioning of the eMMC. For each mirror, DRIS sets a weight coefficient. On a successful boot, the weight coefficient for an appropriate mirror image is dropped. Every unsuccessful boot adds +1 to the weight. If a weight of an image exceeds a threshold, this image is marked as bad and the system tries not to use it in an OS boot path. If all mirrors are bad, then a recovery procedure is initiated. DRIS is seamless to a user.

Determination of an OS boot path is accomplished as follows:

- If a previous OS boot was successful, the current OS boot path is used.
- If a previous boot failed and there was an update, DRIS rolls back (changing mirrors to the last good ones before the update), and essentially OS boot path returns to its previous state. DRIS also increases weights for updated images so that OS recovery can verify and repair images if necessary. If a mirror's weight is too high (greater than bcm_boot_max_count), this image is marked as bad, and is not used anymore for OS boot path by UEFI until it is repaired by boot recovery service (see below).
- If a kernel/dtb image ceases to exist (deleted from ESP, and so on), DRIS assigns a maximum weight and marks the mirror as bad. On successful boot, this image is healed by boot recovery service.
- If there is not update or if the previous version fails, DRIS constructs an OS boot path by searching for mirrors with the least weight. If it is able to find such a mirror, this mirror becomes the current mirror for an appropriate part of OS boot path.
- If DRIS is not able to find a mirror (or all mirrors are bad), a recover boot is forced.

B.2 DRIS Variables

DRIS uses specific non-volatile variables to determine that Linux boot (main, mirror) was successful.

- bcm_dris_support variable:

  This variable indicates that the DRIS scheme is used. DRIS is disabled if this variable is not set.
- bcm_boot_count_N variable:

  This variable determines the last Linux boot status of the current OS boot path. On a successful boot using this boot path, the OS clears this variable. If this variable is set on the next boot (after applying autorun.nsh), UEFI inspects possible reasons and constructs a new OS boot path by using the weight coefficient and mirror status.
- bcm_mboot_count_IMG_N runtime variable:

  This variable is used to determine the weight and status of a certain mirror type image (kernel, dtb, rootfs). This variable is cleared by a successful boot.

B.3 Enabling/Disabling DRIS

DRIS is disabled by default. DRIS can be enabled or disabled via the following UEFI commands:

1. Startup DRIS with the following command:

   ```
   startup dris
   ```

   From Linux:

   ```
   /usr/share/edk2/dris.sh on
   ```

2. DRIS initializes and informs on its status as follows:

   ```
   DRIS is now ON
   ...
   Dual Redundancy Images Support (DRIS)
   DRIS: Current images are healthy
   DRIS: dtb=2; kernel=1; rootfs=1
   DRIS stays ON until

   startup nodris
   ```

3. The following command is issued (for UEFI), or

   ```
   /usr/share/edk2/dris.sh off
   ```

For Linux:

4. DRIS reports that it is turned OFF as follows:

   ```
   DRIS is now OFF
   …
   ```

B.4 Enable/Disable Boot Recovery Service

If `bcm_max_boot_count` is set, the UEFI checks if the current `bcm_boot_count` variable value exceeds `bcm_max_boot_count`. If it does, the UEFI starts the recovery image. The boot recovery service in Linux clears the `bcm_boot_count` variable in memory. If a normal shutdown occurs, this variable is cleared in NV variable storage as well. Otherwise, Linux continues booting until the `bcm_boot_count` value becomes `bcm_mx_boot_count`. In that case, the Linux Boot recovery service initiates an automatic reboot which, if successful, leads to `bcm_boot_count` being cleared. This is done for cases when the power is unexpectedly turned off rather than using the proper shutdown/reboot process.

If the system cannot reboot, `bcm_boot_count` (which was increased automatically by UEFI before booting into Linux) becomes greater than `bcm_max_boot_count`, and the UEFI initiates the recovery image boot.

To force the card to not boot up with the recovery image, use the following instructions to disable the recovery service:

1. Launch an editor or modify the file directly as follows:

   ```
   systemctl edit bootrecovery.service
   ```

2. Set the `bcm_max_boot_count` to 0 in `/etc/systemd/system/bootrecovery.service.d/override.conf` as follows:

   ```
   [Service]
   Environment="BOOTRECOVERY_MAX_BOOT_COUNT=0"
   ```

3. To exit the boot recovery mode, clear `bcm_boot_count` to 0 and reboot to return to normal mode. This can be accomplished via a systemctl reload as follows:

```
systemctl reload bootrecovery.service
reboot
```

Appendix C: SmartNIC Nitro Configurations

The PS225 and PS410T NIC cards come out of manufacturing with a default 8 + 8PF and 8 + 4PF configuration, respectively. The default configurations do not support SR-IOV or pairing models. It is supplied as a common-base config. from manufacturing with the MAIA accessible using IP address 192.168.1.10. These configurations must be updated to access the intelligent features of this 2-port 25G or 4-port 10G Smart NIC adapter.

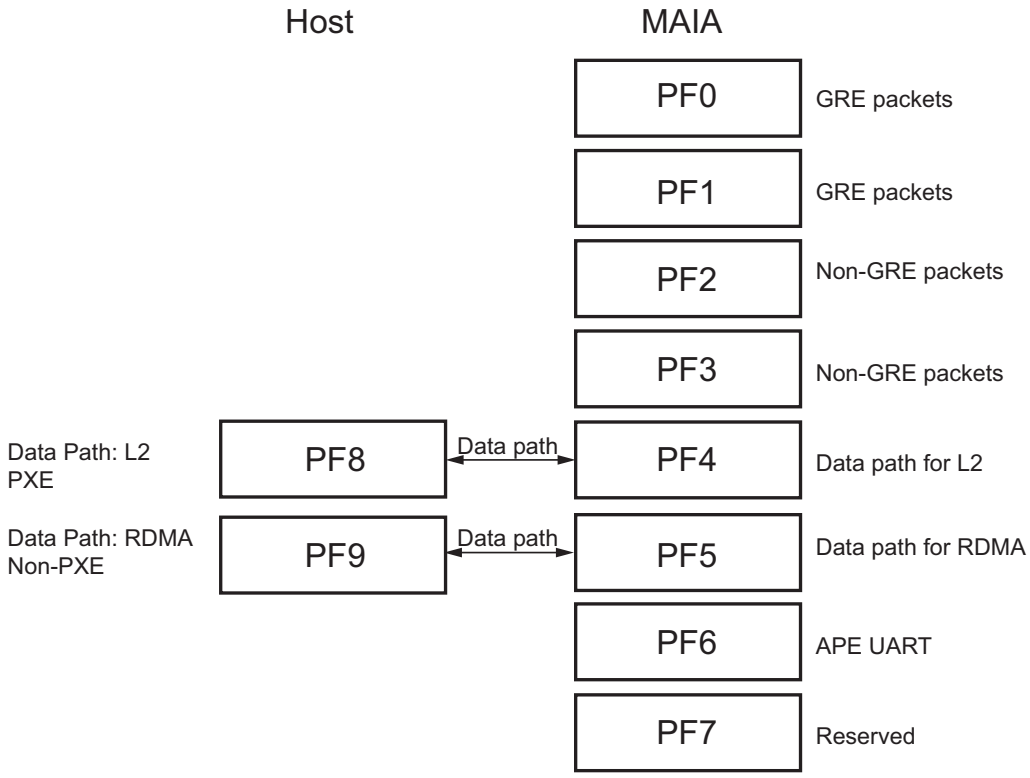
The following configurations allow for up to 64 VF pairs, 128 VF Representor pairs, VirtIO forwarding, and bare metal configurations with GRE traffic. Use the provided tools to upgrade default images to the desired SmartNIC configuration.

C.1 8 + 2 Bare-Metal Configuration

This configuration exposes bare-metal features of the SmartNIC interfaces. The bare-metal system differs in that non-VXLAN traffic is processed by a Linux kernel stack executing on the SmartNIC. Therefore, in the bare-metal system, all traffic is processed by the SmartNIC. The bare-metal system also has additional requirements related to management issues, such as out-of-band management of SmartNIC processors, PXE over VXLAN, and limiting functions that can be performed via host PFs.

The two egress ports of the SmartNIC are port 0 (PF8) and port 1 (PF9). The even-numbered PFs in this configuration are associated with Port 0 and the odd-numbered PFs are associated with Port 1. Therefore, GRE and non-GRE packets are egressed over both physical ports as described in the below image. GRE packets are filtered out and directed to the first pair of PFs while the rest of the traffic is directed towards the second pair of PFs. PF6 is directed towards the out-of-band management firmware running on the APE processor.

Figure 8: 8 + 2 Bare-Metal Configuration

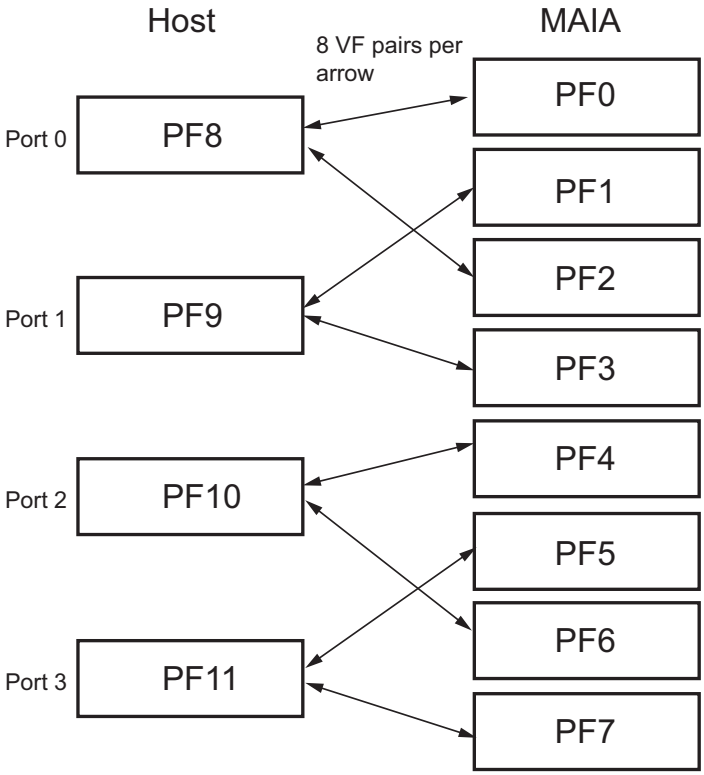


C.2 8 + 4 Pairing Configuration

In the Virtualized System, all traffic that is not VXLAN encapsulated is handled by the host hypervisor in the normal manner (in other words, the traditional non-SmartNIC manner) over a PCIe Physical Function (PF), while all VXLAN-encapsulated traffic is associated with a virtual switch that executes on the SmartNIC.

This configuration exposes physical function and virtual function pairing of the SmartNIC interfaces. The two egress ports of the SmartNIC are port 0 (PF8, PF10) and port 1 (PF9, PF11). The even-numbered PFs in this configuration are associated with Port 0 and the odd-numbered PFs are associated with Port 1. Each MAIA PF is capable of configuring 8 VFs with 16 queues each. Each host PF is capable of supporting 16 VFs. With this configuration, PF-to-PF pairing as well as VF-to-VF pairing is possible with up to 4 PF pairs or 64 VF pairs where four sets of 16-host VFs are directed to eight sets of 8 VFs on MAIA.

Figure 9: 8 + 4 Pairing Configuration

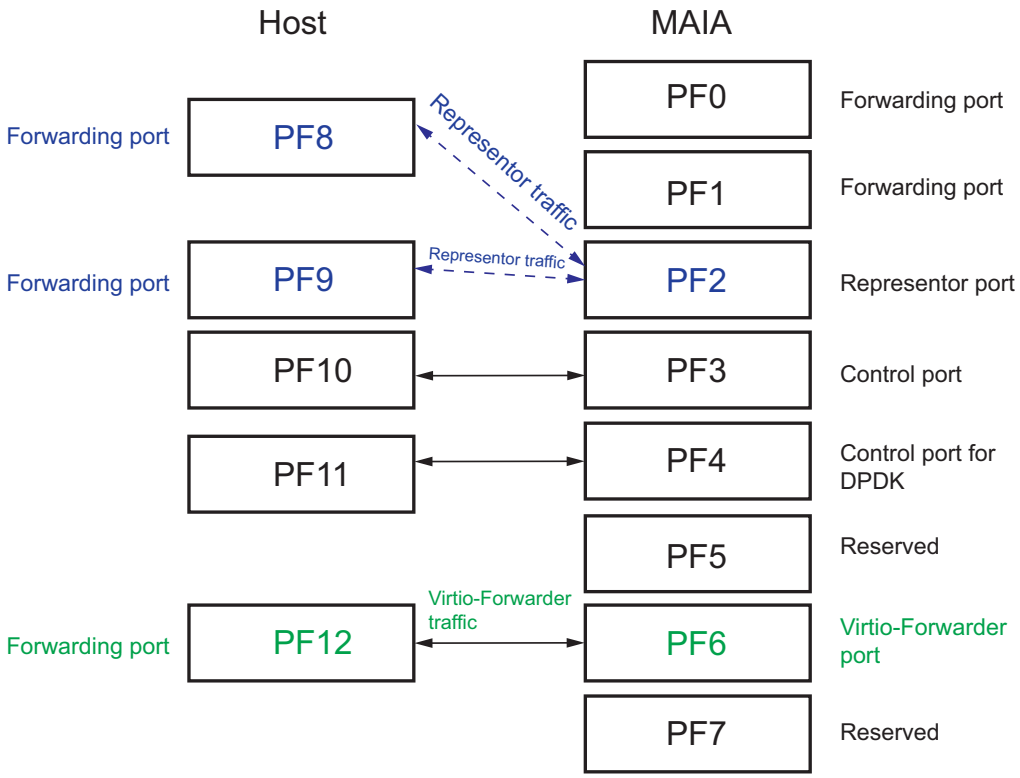


C.3 8 + 5 Representor Configuration

This configuration exposes 128 Representor pairings of the SmartNIC interfaces and/or a VirtIO forwarding path, although configuring both prevents the full 128 Representors from being instantiated due to resource availability.

The two egress ports of the SmartNIC are port 0 (PF8) and port 1 (PF9) and two PF pairs (PF10, PF11) for control packet and dpdk control packets. On MAIA, PF2 is the mux/demux point for the 128 Representors split evenly between both Host forwarding ports before egressing the traffic over the SmartNIC ports. The VirtIO traffic is being forwarded over a PF pair (PF6-PF12) as an alternative method of traffic management instead of the representor pair model.

Figure 10: 8 + 5 Representor Configuration



Stingray SmartNIC platform software consists of the following pre-generated Nitro config profiles, part (`nitro/extra_cfgs/` directory) of image tarball package.

For 10G Speed:

```
bcm958802a8044_1x10g.pkg
bcm958802a8044_2x10g.pkg
bcm958802a8044_2x10g_8+2_baremetal.pkg
bcm958802a8044_2x10g_8+3_pf.pkg
bcm958802a8044_2x10g_8+4_pf.pkg
bcm958802a8044_2x10g_8+4_pf_rep.pkg
bcm958802a8044_2x10g_8+5_pf_rep.pkg
```

For 25G Speed:

```
bcm958802a8044_1x25g.pkg
bcm958802a8044_2x25g.pkg
bcm958802a8044_2x25g_8+1_baremetal.pkg
bcm958802a8044_2x25g_8+2_baremetal_nofec.pkg
bcm958802a8044_2x25g_8+2_baremetal.pkg
bcm958802a8044_2x25g_8+3_pf.pkg
bcm958802a8044_2x25g_8+4_pf.pkg
bcm958802a8044_2x25g_8+4_pf_rep.pkg
bcm958802a8044_2x25g_8+5_pf_rep.pkg
```

For Autoneg Link Speed:

```
bcm958802a8044_1xAN.pkg
bcm958802a8044_2xAN.pkg
bcm958802a8044_2xAN_8+1_baremetal.pkg
bcm958802a8044_2xAN_8+2_baremetal.pkg
bcm958802a8044_2xAN_8+3_pf.pkg
bcm958802a8044_2xAN_8+4_pf.pkg
bcm958802a8044_2xAN_8+4_pf_rep.pkg
bcm958802a8044_2xAN_8+5_pf_rep.pkg
```

For RDMA Support:

```
bcm958802a8044_8+8_pf_rdma.pkg
```

Miscellaneous/Generic:

```
BCM958802A8044C.pkg
stingray_1p.pkg
stingray_2p.pkg
stingray_4p.pkg
```

Nitro Config File Naming Format:

```
bcm958802a80<platform/card type>_<number of ports>x<link config type>_<Maia PFs>+<Host PFs>_[if
baremetal config]_pf_[representor support]_[miscellaneous support].pkg
```

Examples:

- File: bcm958802a8044_2x25g_8+2_baremetal_nofec.pkg

  This is for card/platform PS225-4GB, 2 ports, 25G forced speed link, 8 PFs on the MAIA host, 2 PFs on host (x86), baremetal config, and no link FEC enabled.
- File: bcm958802a8044_2xAN_8+4_pf_rep.pkg

  This is for card/platform PS225-4GB, 2 ports, Autoneg, 8 PFs on MAIA host, 4 PFs on host(x86), representor pair model config.

Table 5: Card/Platform Type

bcm958802a8044	PS225 – 4 GB
bcm958802a8048	PS225 – 8 GB
bcm958802a8046	PS225 – 16 GB
bcm958802a8023	PS410T – 4 GB

Additional Properties:

Table 6: Speed, Autoneg, Autodetect, and FEC

Nitro Config: .pkg File	Speed	Autodetect	FEC
bcm958802a8044_2x10g_[anything].pkg	10G forced speed	N/A	Disabled
bcm958802a8044_2x25g_[anything].pkg	25G forced speed	N/A	Disabled
bcm958802a8044_2xAN_[anything].pkg	10G or 25G	Enabled	N/A

Table 7: Number of PFs and VFs

Nitro Config: .pkg File	MAIA PFs	Host PFs	VFs
<platform>_<port config>_8+1_baremetal.pkg	8	1	N/A
<platform>_<port config>_8+2_baremetal.pkg	8	2	0 on MAIA and 128 on x86 (64 each on PF8 and PF9 on x86)
<platform>_<port config>_8+3_pf.pkg	8	3	0 on MAIA and 128 on x86 (64 each on PF8 and PF9 on x86)
<platform>_<port config>_8+4_pf.pkg	8	4	64 on MAIA and 64 on x86 (8 on each MAIA PF and 16 on each x86 PF)
<platform_<port config>.pkg	8	8	N/A

Table 8: Representor Pair on MAIA Host to Support Max (128) VFPairs on Host (x86) Pairs

Nitro Config: .pkg File	MAIA PFs	Host PFs	VFs	Representor Pair Support?
<platform>_<port config>_8+4_pf_rep.pkg	8	4	128 VFs on x86(First two PFs supports 64 VFs each)	Yes
<platform>_<port config>_8+5_pf_rep.pkg	8	5	128 VFs on x86(First two PFs supports 64 VFs each)	Yes

Appendix D: Frequently Asked Questions

Q: How can you check the current version of the software installed on card?

A: Check the scripts in yocto rootfs under directory `/usr/share/edk2/` and `/usr/share/edk2/version/`

Scripts list (at /usr/share/edk2/):

```
bootrecovery.sh       dris.sh           efivars.sh      force_recovery.sh     image_update.sh
show_bootlog.sh      ubuntu-chroot.sh  yocto2ubuntu.sh
check_elog_en.sh     dump_efivars.sh    error_codes.sh  force_repartition.sh  inband_network.sh
show_rootfs_part.sh  version
check_stingray_rev.sh dump_nitro_freq.sh force_normal.sh get_upgrade_status.sh repartition.sh
tftp_update.sh       yocto2centos.sh
```

Scripts list (at /usr/share/edk2/version/):

```
get_cur_version.sh  get_modules_ver.sh  get_nitro_fw_ver.sh  get_nitro_nvm_ver.sh
show_mcu_patch_ver.sh  show_release_ver.sh  show_uefi_ver.sh
```

To view the driver and firmware version of the network ports:

```
ethtool -i <interface name>
```

Q: How can you pass additional arguments to kernel bootargs?

A: To pass an additional argument to kernel boot argument list, stop in UEFI and set `extraarg` variable using the following example command :

Syntax:

```
Shell> set extraarg "<arguments>"
> startup auto
> startup boot
```

Example:

```
set extraarg "Pie_aspm=off"
```

Once in the Linux shell, verifying with the following:

```
cat /proc/cmdline
```

Q: How do you disable the recovery boot image?

A: See Enable/Disable Boot Recovery Service.

Q: What do you check when network connectivity (ping and such traffic) is not working?

A: Ensure the following check to make basic setup correct.

1. The PS225 does not support a combination of different speeds on ports. For example, if port-1 links up at 25G and port-2 links up at 10G this is not supported.
2. The correct cable must be used for a 25G port. Connecting a 10G cable while leaving the unconnected port running at 25G by default causes a scenario combination of port-x at 10G and port-y at 25G, which is not supported.
3. Check the peer device config thoroughly. The SmartNIC can be used in default factory mode (Auto-negotiation ON) to link up with peer device configs.
4. Ensure that there is not a duplicate IP address configuration.
5. Ensure that no more than one IP address in the same subnet. Configuring two different interface IP addresses in the same subnet does not work.

Q: How to check if the PS225 silicon on card is Ax or Bx?

A: Use one of the following methods:

1. Use the following script which is available on the PS225:

   ```
   root@bcm958802a8048c:~# cat /usr/share/edk2/check_stingray_rev.sh
   ```

2. Use lspci command output to determine the paxc (PCI bridge) root complex Pci ID.
   - For Bx it is 750.
   - For Ax it is the same as pf0 (16f0, d802, and so forth.).

NOTE: If your card is old, it may have Ax silicon. Ax silicon is no longer supported in the latest software. Contact Broadcom product support before performing software image upgrade.

Revision History

5880X-PS225-UG108; December 22, 2020

Updated:

- Table 2, eMMC Partitions
- Boot the Recovery Image
- Setting Up Ubuntu on a PS225
- Booting the Ubuntu/CentOS/other Distro rootfs Image

Added:

- Booting the Ubuntu/CentOS/other Distro rootfs Image

5880X-PS225-UG107; January 31, 2020

Updated:

- PS225 Card Variants

5880X-PS225-UG106; October 8, 2019

Added:

- Booting Different Images

5880X-PS225-UG105; July 10, 2019

Updated:

- Setting Up the Serial Console (Optional)

5880X-PS225-UG104; May 24, 2019

Added:

- Appendix C, SmartNIC Nitro Configurations

5880X-PS225-UG103; March 1, 2019

Added:

- Dual Redundancy Images Support (DRIS)

5880X-PS225-UG102; February 1, 2019

Added:

- Appendix A, Repartitioning the eMMC for Larger Data Partitions

5880X-PS225-UG101; December 12, 2018

Updated:

- Upgrading the PS225 Software

5880X-PS225-UG100; October 4, 2018

Initial release.


BROADCOM®